

Voice Agent Latency & Reliability Observatory

Benchmarking, Latency Measurement, Model Comparison and Observability for AI Voice Pipelines

Author: ValikalaChandra

Institution: Not documented in the repository

Date: September 2026

This report summarizes the final repository state. It is based on the checked-in implementation, stored benchmark artifacts, screenshots, audit reports, and test documentation. It does not invent missing measurements.

Phase 1 - Understand and Define the Problem

The project is a controlled observatory for AI voice-agent pipelines. A recorded WAV file is processed by STT, passed into a LangGraph agent and LLM, optionally routed through a tool, and converted back to speech through TTS.

The headline metric is TTFA = timestamp(first output audio) - timestamp(end of input audio). Request start, upload start, LLM first token, and TTS completion are different boundaries and are not substituted for TTFA.

Phase 2 - Design the Architecture

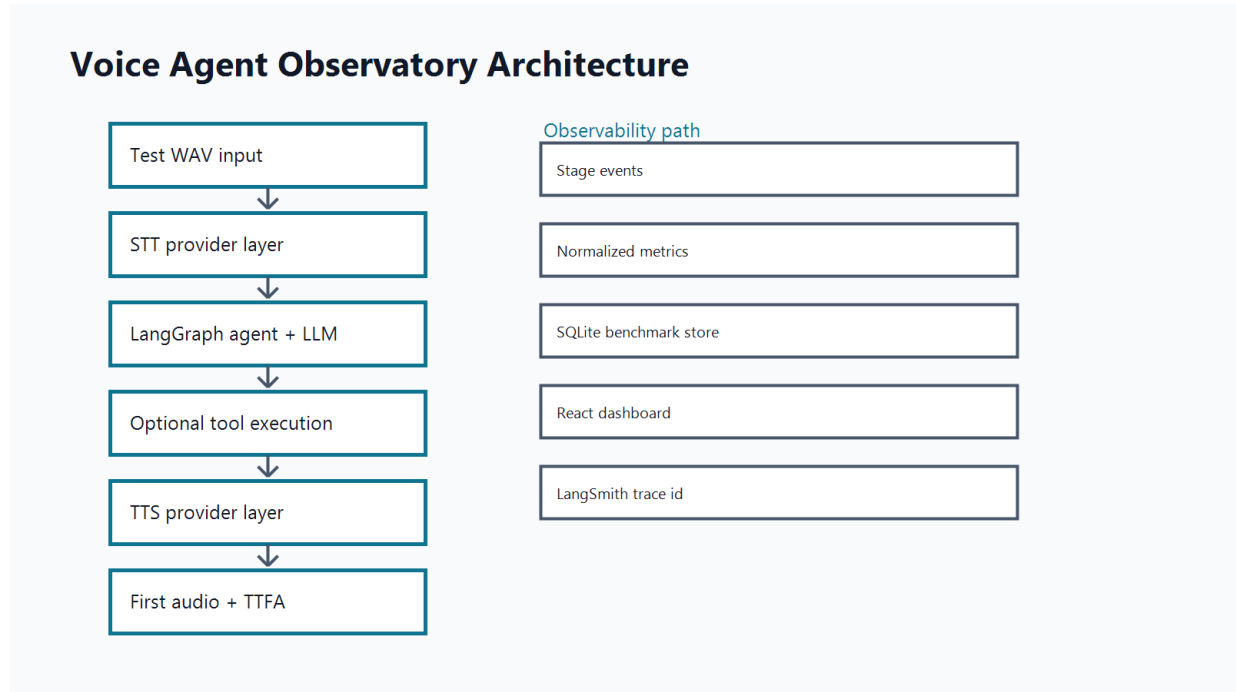


Figure. Architecture overview: WAV input, provider layers, LangGraph, optional tools, TTS, storage, dashboard, and LangSmith observability.

- Frontend: React/Vite dashboard for test cases, configuration, live runs, result detail, comparison, reliability, and concurrency.
- Backend: FastAPI REST/WebSocket API, provider availability, benchmark orchestration, exports, and concurrency tests.
- Pipeline runner: single timing authority for INPUT_END, STT, LangGraph/LLM, tools, TTS, FIRST_AUDIO, and persistence.
- Storage and observability: SQLite rows, JSON artifacts, live WebSocket events, screenshots, and optional LangSmith traces.

Phase 3 - Model Plug-in Architecture

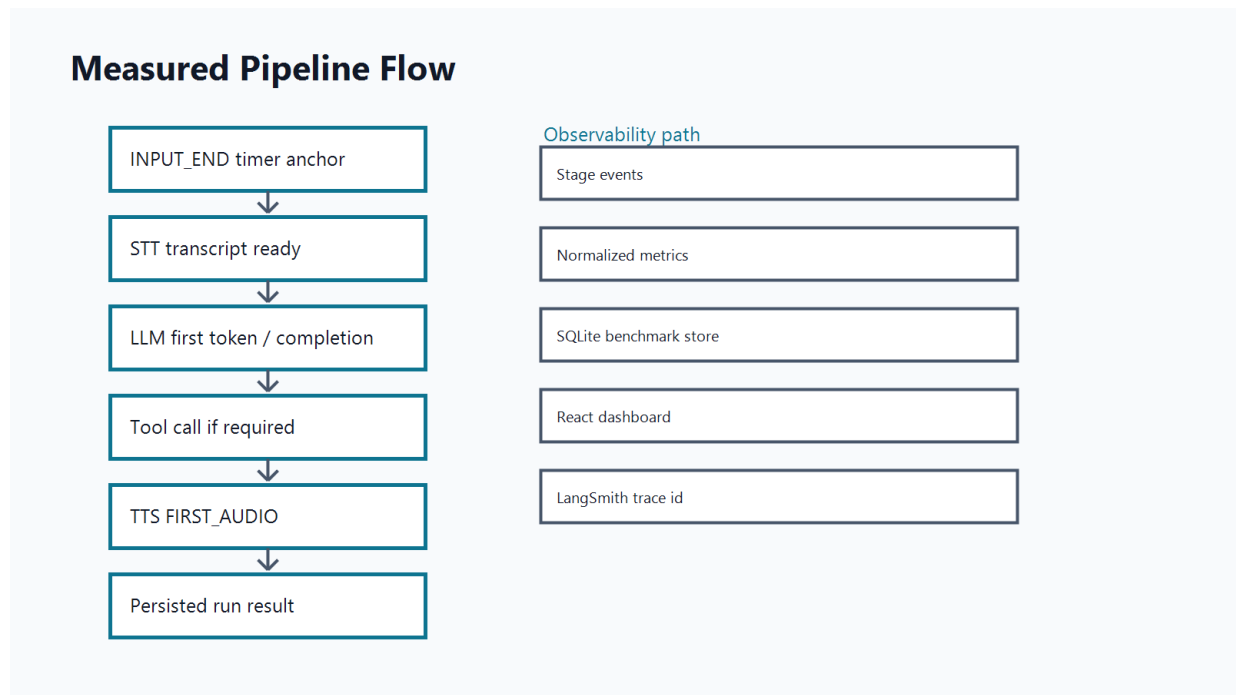


Figure. Measured pipeline flow with TTFA anchored at INPUT_END and stopped at the first emitted audio bytes.

Stage	Provider	Models	Status
STT	Deepgram	nova-2, nova-3, hosted whisper-large	Verified
STT	ElevenLabs	scribe_v1	Verified earlier; quota/auth limited later
TTS	Deepgram	aura-2-thalia-en, aura-2-andromeda-en	Verified
TTS	ElevenLabs	turbo, multilingual, flash	Verified earlier; quota limited later
TTS	OpenAI	tts-1	NOT_CONFIGURED; not faked
LLM	OpenRouter	configured chat model	Verified

Selections use provider:model[:voice] specs, are validated against allowlists, and are persisted per run. Missing keys mark providers unavailable rather than falling back silently.

Phase 4 - Benchmark Execution

Case	Description	Expected tools	Noise
TC01	Direct spoken query	0	none
TC02	Direct spoken query	0	none
TC03	Weather query	1	none
TC04	Web search query	1	none
TC05	Noisy knowledge question	0	environmental
TC06	Noisy direct query	0	environmental

The repository contains 415 benchmark JSON artifacts. Some named final experiment aggregate files are not present as standalone EXP-FINAL*.json files; raw per-run artifacts remain under docs/benchmark-results/.

Phase 5 - Latency Measurement

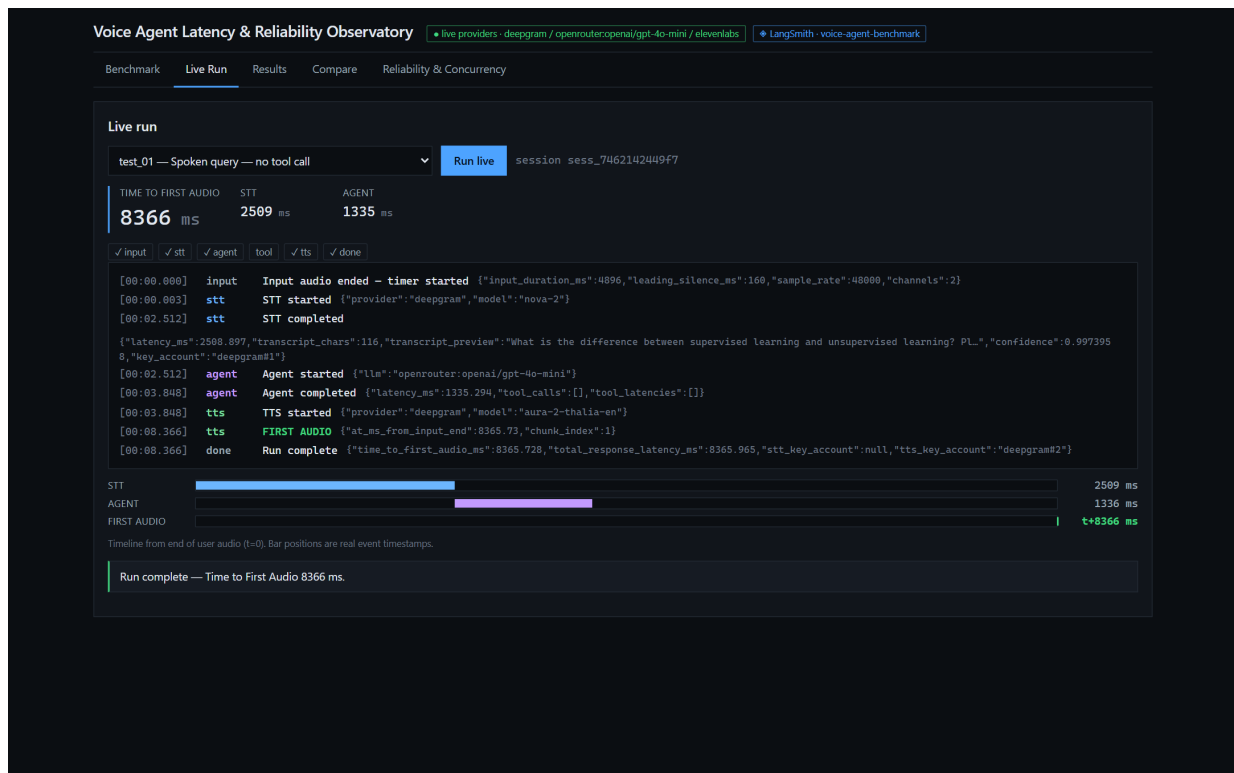


Figure. Latency waterfall screenshot showing stored stage events and FIRST_AUDIO timing.

- STT latency: provider call start to transcript ready.
- LLM TTFT/completion: LangGraph agent start to first token/final response.
- Tool latency: measured tool body execution.
- TTS first-audio latency: TTS start to first emitted audio chunk.
- TTFA: INPUT_END to FIRST_AUDIO.
- Total response latency: INPUT_END to TTS completion.

Phase 6 - Compare Models

STT	LLM	TTS	Test	TTFA
Deepgram nova-2	gpt-4.1-mini	Aura Thalia	TC01	9,582 ms
Deepgram nova-3	gpt-4.1-mini	Aura Thalia	TC01	8,811 ms
Deepgram whisper-large	gpt-4.1-mini	Aura Thalia	TC05	8,910 ms
Deepgram nova-3	gpt-4.1-mini	Aura Andromeda	TC01	7,474 ms
Deepgram nova-2	gpt-4.1-mini	ElevenLabs turbo	TC01	TTS quota failure

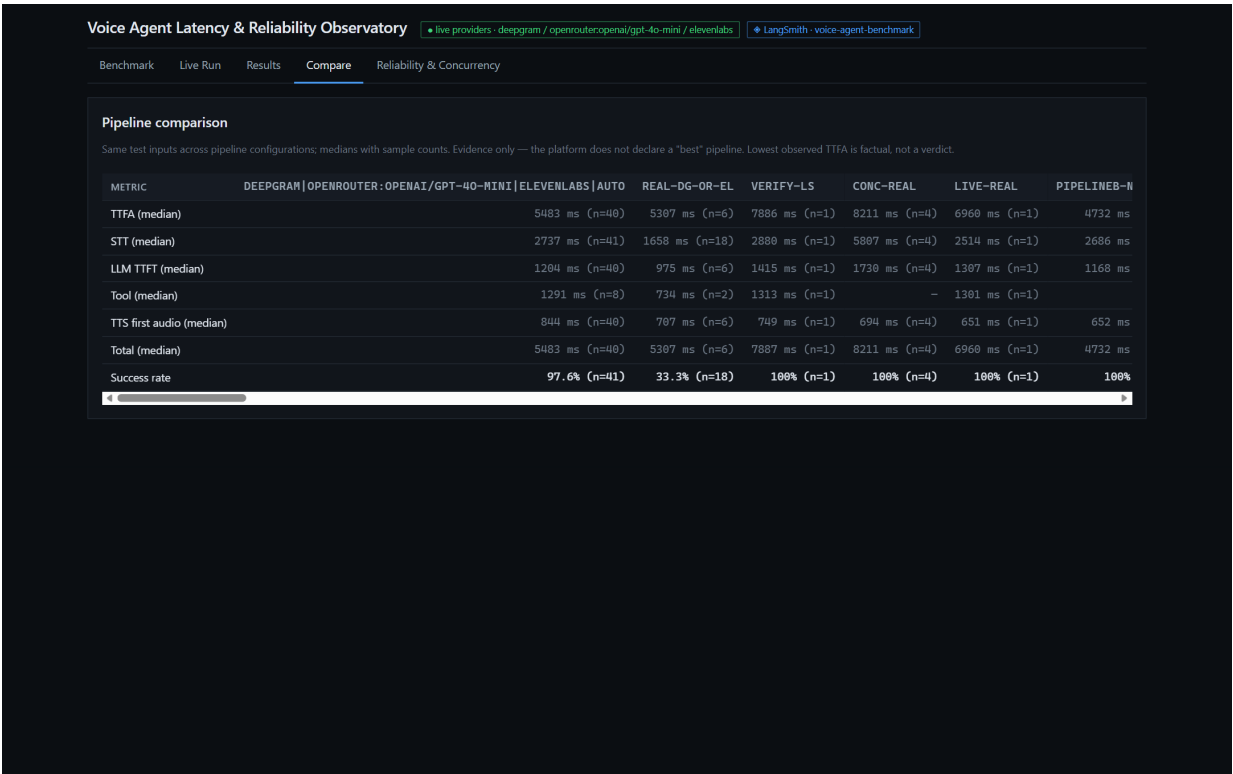


Figure. Model comparison dashboard using stored run counts and latency metrics.

The earlier EXP-2026-00003 report measured gpt-4.1-mini median TTFA 6,093 ms over 12 runs and gemini-3.8-flash median TTFA 6,945 ms over 12 runs with shared STT/TTS. It also exposed gemini weather-tool over-calling on TC03.

Phase 7 - Reliability Testing

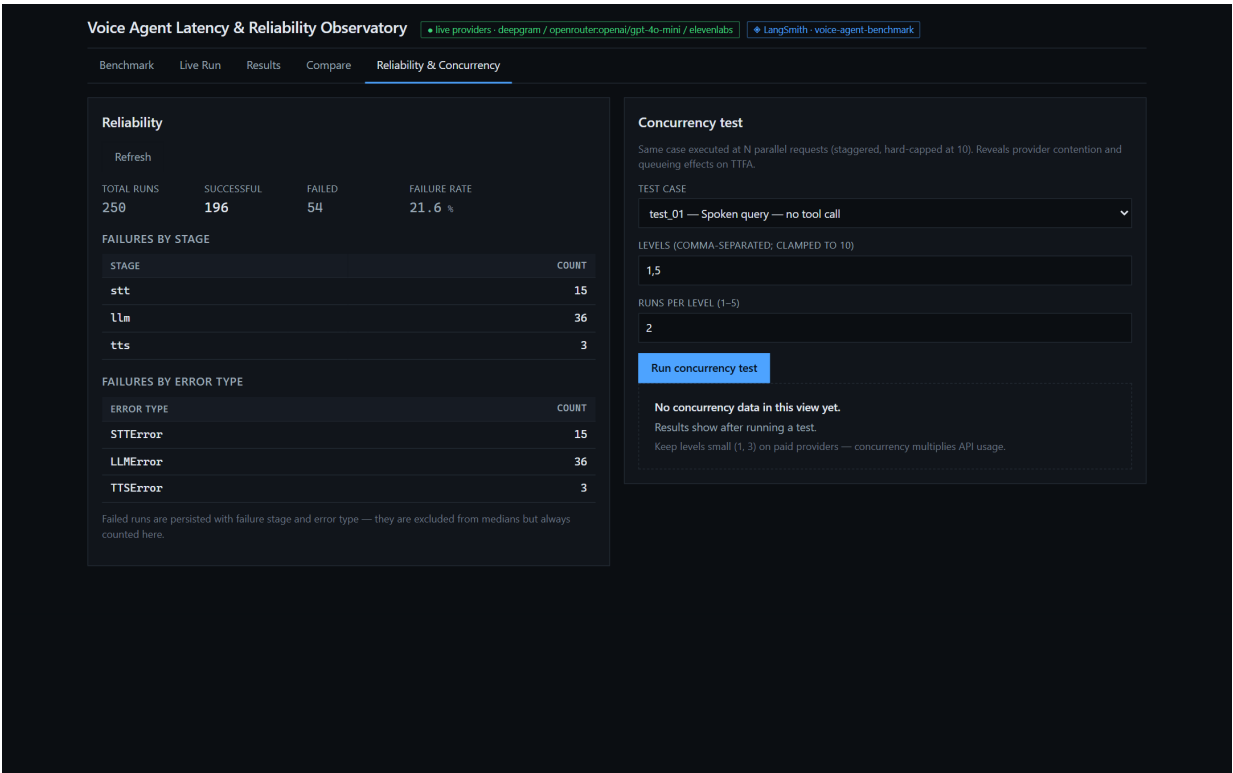


Figure. Reliability dashboard with stage-tagged failure counters.

Reliability testing covers provider timeouts, quota failures, invalid audio, tool behavior, LLM failures, TTS failures, and safe error visibility. Failures retain stage, error type, safe detail, provider specs, and timestamps.

Phase 8 - Basic Concurrency Test

Level	Success	TTFA median	Throughput
1 real stack	2/2	5,913 ms	0.30 rps
5 real stack	10/10	6,896 ms	1.16 rps
10 real stack	Not claimed	Not available in captured run	Not available

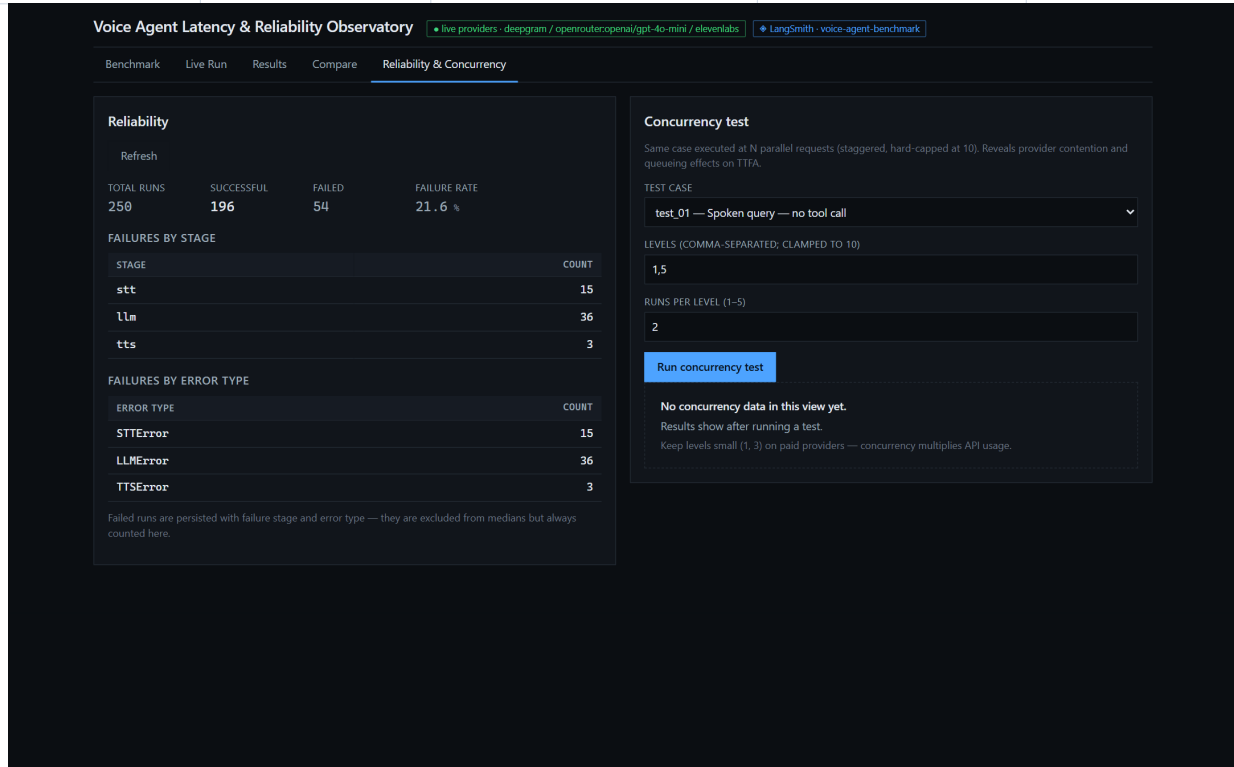


Figure. Concurrency panel with capped controls to protect provider quotas.

Phase 9 - Frontend / Observability Dashboard

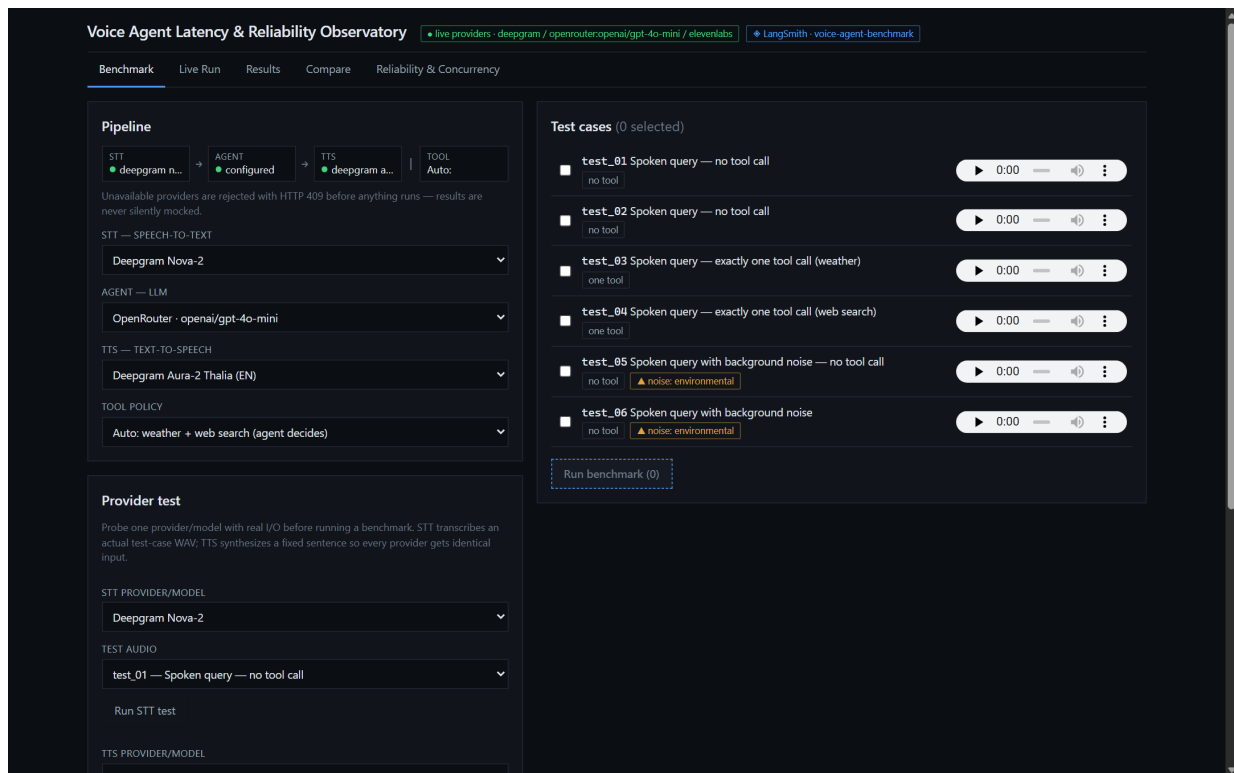


Figure. Dashboard overview emphasizing operational status and recent evidence.

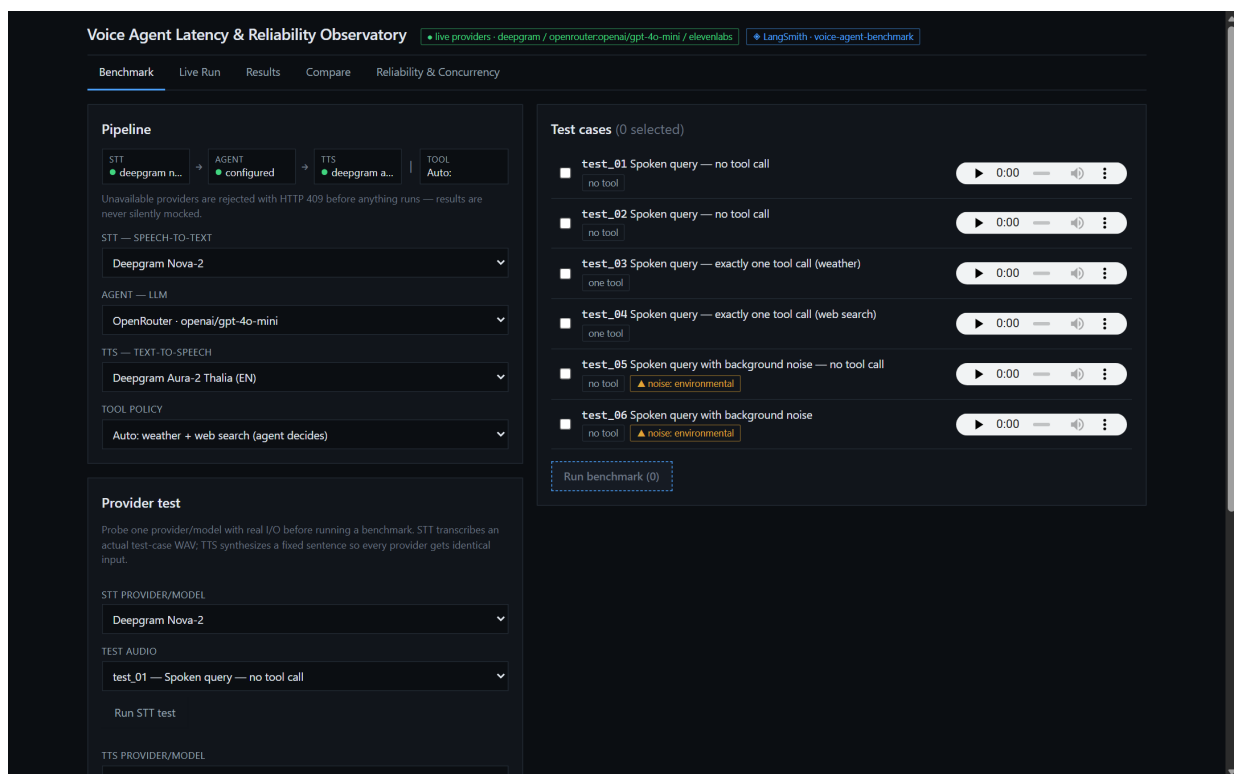


Figure. Pipeline configuration screen with executable provider/model specs.

Phase 10 - Final Summary

The repository contains a working voice-agent observatory with configurable STT, LLM, and TTS stages; event-level instrumentation; TTFA measurement from INPUT_END to FIRST_AUDIO; LangGraph execution; optional tool timing; LangSmith trace IDs; SQLite persistence; model comparison; reliability and concurrency views; security

controls; and audit documentation.

Known limitations are explicit: ElevenLabs free-tier quota exhaustion after earlier verification, OpenAI TTS not configured, hosted whisper cold starts, buffered first-audio semantics for current working TTS providers, no auth for the local dashboard, and no claimed real-stack level-10 concurrency run.

Phase 11 - Source Code

Repository URL: <https://github.com/Sudesh-chandra/VoiceAgentDashboard.git>

- backend/: FastAPI API, pipeline runner, provider adapters, LangGraph agent, storage, tests, and scripts.
- frontend/: React/Vite dashboard.
- data/test_cases/: six imported WAV test cases and metadata.
- docs/: architecture, assumptions, model reports, audit reports, benchmark artifacts, and this report.
- screenshots/: captured dashboard evidence.
- AI_USAGE.md: AI assistance and engineering decision log.

Reproduction summary: configure .env from .env.example without committing secrets; run backend tests in MOCK_MODE; start FastAPI and Vite; run benchmarks through the dashboard or backend/benchmark/run_matrix.py.